CS636 Final Term Preparation

Mubeen Mustafa

BC200403880

cs636 Quiz No 1

Question # 1 of 10 (Start time: 04:16:37 PM, 02 May 2024) Total Marks: 1

There are _____ tuples in finite state machine.

Select the correct option

4

3

2

5 Correct

Question # 2 of 10 (Start time: 04:16:54 PM, 02 May 2024) Total Marks: 1

Formal specifications allow for feedback on system specifications at _____ development stages.

Select the correct option

early Correct

middle

early and later

later

Question # 3 of 10 (Start time: 04:17:12 PM, 02 May 2024) Total Marks: 1

In petri nets, place is denoted by _____.

Select the correct option

circle Correct

square

arrow

oval

Question # 4 of 10 (Start time: 04:17:36 PM, 02 May 2024) Total Marks: 1

Correctness of entire software depends on _____.

Select the correct option

Maintenance

Testing

Specification Correct

Deployment

Question # 5 of 10 (Start time: 04:17:54 PM, 02 May 2024) Total Marks: 1

In petri nets, tokens are denoted by a _____.

Select the correct option

filled circle Correct

directed arrow

rectangle

vertical bar

Question # 6 of 10 (Start time: 04:18:10 PM, 02 May 2024) Total Marks: 1

The minimum number of states required to recognize an octal number divisible by 3 are/is

Select the correct option

2

3 Correct

1

5

Question # 7 of 10 (Start time: 04:18:26 PM, 02 May 2024) Total Marks: 1

_____ constitute a branch of software engineering that incorporates the use of mathematics for software development.

Select the correct option

Formal methods Correct

Requirement engineering

Quality assurance

Project management

Question # 8 of 10 (Start time: 04:18:48 PM, 02 May 2024) Total Marks: 1

How many elements in the set for the Language L={xϵ(∑r) *|length if x is at most 2} and ∑={0,1} is_____

Select the correct option

8

6

5

7 Correct

Question # 9 of 10 (Start time: 04:19:59 PM, 02 May 2024) Total Marks: 1

A _____ model that represents the intended behavior of a program can be used as a formal specification.

Select the correct option

mathematical Correct

data

conceptual

computational

Question # 10 of 10 (Start time: 04:20:27 PM, 02 May 2024) Total Marks: 1

The language that can be expressed by any regular expression is called a _____

Select the correct option

Binary language

Prime Language

Factorial

Regular language Correct

CS636 Quiz No 2

Question # 1 of 10 (Start time: 09:48:15 AM, 14 May 2024) Total Marks: 1

g0 represents:

Select the correct option

set of actions

initial location

effect function

initial condition Correct

Question # 2 of 10 (Start time: 09:50:06 AM, 14 May 2024) Total Marks: 1

Which of the following is NOT a phase of model checking process

Select the correct option

Modeling phase

Integrated phase Correct

Analysis phase

Running phase

Question # 3 of 10 (Start time: 09:50:48 AM, 14 May 2024) Total Marks: 1

Modeling phase formalize the property to be checked using the property ……..

Select the correct option

Description language

General language

Specification language Correct

Binary Language

Question # 4 of 10 (Start time: 09:51:23 AM, 14 May 2024) Total Marks: 1

A model or a diagram is well-formed if its structure satisfies the _____ rules.

Select the correct option

development

study

syntactic Correct

system

Question # 5 of 10 (Start time: 09:51:54 AM, 14 May 2024) Total Marks: 1

Which one is the correct representation of effect function?

Select the correct option

Effect : Act - Eval(Var) → Eval(Var)

Effect : Act + Eval(Var) → Eval(Var)

Effect : Act / Eval(Var) → Eval(Var)

Effect : Act × Eval(Var) → Eval(Var) Correct

Question # 6 of 10 (Start time: 09:52:27 AM, 14 May 2024) Total Marks: 1

Select the disadvantage of using Model Checking.

Select the correct option

It supports partial verification

It is a potential “push-button” technology

It checks only stated requirements Correct

Model checking is particularly valuable for verifying concurrent systems

Question # 7 of 10 (Start time: 09:53:12 AM, 14 May 2024) Total Marks: 1

Generally speaking, a consistency constraint C is a predicate defined on models such that _____ means that the model is consistent with respect to the constraint.

Select the correct option

M(C) = true

C(M) = false

M(C) = false

C(M) = true Correct

Question # 8 of 10 (Start time: 09:53:56 AM, 14 May 2024) Total Marks: 1

The executable actions of a _____ system typically result from conditional branching.

Select the correct option

user-dependent

data-dependent Correct

tool-dependent

cost-dependent

Question # 9 of 10 (Start time: 09:54:34 AM, 14 May 2024) Total Marks: 1

A transition system TS is a tuple (S, Act,→,I, AP, L) where S is a _____.

Select the correct option

set of actions

set of location

set of states Correct

effect function

Question # 10 of 10 (Start time: 09:55:29 AM, 14 May 2024) Total Marks: 1

Act represents:

Select the correct option

effect function

set of actions Correct

set of locations

initial condition

CS636 Quiz No 3

Question # 1 of 10 (Start time: 06:25:27 PM, 20 May 2024) Total Marks: 1

A proposition is said to be …….. if it cannot be broken down into simpler propositions, that is, if it is not composite.

Select the correct option

Command

Logical

Conditional

Primitive Correct

Question # 2 of 10 (Start time: 06:25:49 PM, 20 May 2024) Total Marks: 1

An argument is an assertion that a given set of propositions P1, P2,...,Pn, called …..

Select the correct option

Premises Correct

Solution

Conclusion

Contradictions

Question # 3 of 10 (Start time: 06:26:09 PM, 20 May 2024) Total Marks: 1

p → q is logically equivalent to _____

Select the correct option

p ∨ ¬q

¬p ∨ q Correct

¬p ∨ ¬q

¬p ∧ q

Question # 4 of 10 (Start time: 06:26:40 PM, 20 May 2024) Total Marks: 1

A compound proposition that is always TRUE and contain only T in their evaluation is called….

Select the correct option

Contingency

Tautologies Correct

Proposition

Contradictions

Question # 5 of 10 (Start time: 06:27:05 PM, 20 May 2024) Total Marks: 1

Let P: This is a great restaurant, Q: You should not come back here. Then ‘This is a great restaurant and you should come back here.’ is best represented by?

Select the correct option

~P V ~Q

P ∧ ~Q

P ∧ Q Correct

P V Q

Question # 6 of 10 (Start time: 06:28:20 PM, 20 May 2024) Total Marks: 1

The universal quantifier denoted by ---------

Select the correct option

∀ Correct

∈

~

∧

Question # 7 of 10 (Start time: 06:28:40 PM, 20 May 2024) Total Marks: 1

“if you work hard you will succeed “ – is an example for _____ proposition.

Select the correct option

Positive

Emotive

Negative

Conditional Correct

Question # 8 of 10 (Start time: 06:28:58 PM, 20 May 2024) Total Marks: 1

Proposition which is affirmed on the basis of premises is called …….

Select the correct option

Conclusion Correct

Idea

Concept

Suggestion

Question # 9 of 10 (Start time: 06:29:19 PM, 20 May 2024) Total Marks: 1

Which of the following is NOT an example of a proposition?

Select the correct option

Ice floats on the water

Where are you going? Correct

2+6=9

2+2=4

Question # 10 of 10 (Start time: 06:29:43 PM, 20 May 2024) Total Marks: 1

p → q is logically equivalent to _____

Select the correct option

p ∨ ¬q

¬p ∧ q

¬p ∨ ¬q

¬p ∨ q Correct

CS636 Quiz No 4

Question # 1 of 10 (Start time: 10:37:34 AM, 26 June 2024) Total Marks: 1

The _____ operator checks whether or not a particular element is present in a particular set.

Select the correct option

assignment

membership (Correct)

subset

cardinality

Question # 2 of 10 (Start time: 10:38:04 AM, 26 June 2024) Total Marks: 1

R represents:

Select the correct option

natural numbers

real numbers (Correct)

characters

integers

Question # 3 of 10 (Start time: 10:38:22 AM, 26 June 2024) Total Marks: 1

A subrange returns the set of all numbers from the _____ to the last number inclusive.

Select the correct option

fourth

first (Correct)

second

third

Question # 4 of 10 (Start time: 10:38:51 AM, 26 June 2024) Total Marks: 1

The invariant definition uses the keyword _____.

Select the correct option

types

inv (Correct)

init

invar

Question # 5 of 10 (Start time: 10:39:15 AM, 26 June 2024) Total Marks: 1

In VDM-SL, the state refers to the _____ data that must be stored by the system.

Select the correct option

complex

permanent (Correct)

partial

temporary

Question # 6 of 10 (Start time: 10:39:38 AM, 26 June 2024) Total Marks: 1

The _____ operator of VDM-SL (card) returns the number of elements in a given set.

Select the correct option

assignment

cardinality (Correct)

membership

subset

Question # 7 of 10 (Start time: 10:40:11 AM, 26 June 2024) Total Marks: 1

A quote type defines a single value, and at the same time defines a _____ containing just that value.

Select the correct option

constant

function

type (Correct)

formula

Question # 8 of 10 (Start time: 10:41:57 AM, 26 June 2024) Total Marks: 1

The purpose of _____ function is to check whether an integer value, val, is within the range MIN and MAX as defined earlier.

Select the correct option

onRange

atRange

inRange (Correct)

ofRange

Question # 9 of 10 (Start time: 10:43:16 AM, 26 June 2024) Total Marks: 1

Subranges used when a set of _____ integers is required

Select the correct option

long

continuous (Correct)

double

short

Question # 10 of 10 (Start time: 10:44:57 AM, 26 June 2024) Total Marks: 1

The _____ of j and k is the set that contains all the elements that belong to j but do not belong to k.

Select the correct option

union

complement

difference (Correct)

intersection

CS636 Sample Paper Solved

Paper Time [90 minutes]

CS636 – Formal Methods

CS636 (Final Exam Sample Paper)

MCQs

Take an assessment.

1- What is the primary purpose of formal methods in software engineering?

a. Code optimization

b. Enhancing user experience

c. Rigorous reasoning and verification

d. Prototyping

2- Formal methods are used for:

a. Informal reasoning

b. Rigorous reasoning

c. Trial and error

d. Random testing

3- Which of the following is a key benefit of using formal methods?

a. Faster development speed

b. Improved code aesthetics

c. Increased likelihood of error detection

d. Reduced need for documentation

4- Verification in software engineering is concerned with:

a. Code execution speed

b. Confirming that a system meets its specifications

c. User interface design

d. Network security

5- Which mathematical model is commonly used for specifying software requirements?

a. UML

b. DFD

c. Finite State Machines

d. Petri Nets

6- The primary goal of software specification is to:

a. Enhance code readability

b. Describe how the software will be implemented

c. Provide a clear and unambiguous description of the system's behavior

d. Increase development speed

7- What characterizes a concurrent system?

a. A system that runs on a single processor

b. A system with multiple independent processes

c. A system without any processes

d. A system with real-time capabilities

8- Well-formedness in concurrent systems refers to:

a. The absence of errors

b. The efficient use of resources

c. Compliance with defined rules or syntax

d. Rapid execution speed

9- In the context of concurrent systems, what does deadlock refer to?

a. Efficient resource utilization

b. A state where processes are unable to proceed

c. Smooth system execution

d. Dynamic process allocation

10- Propositional logic deals with:

a. Objects and relationships between them

b. Statements that are either true or false

c. Mathematical operations

d. Natural language semantics

11- What is a proposition in logic?

a. A statement that can be proven

b. A statement that is always false

c. A statement that is either true or false

d. A statement that is difficult to understand

12- Predicate logic extends propositional logic by introducing:

a. Variables and quantifiers

b. Conditional statements

c. Logical operators

d. Propositions

13- VDM-SL stands for:

a. Visual Design Model - Specification Language

b. Virtual Development Method - Specification Logic

c. Vienna Development Method - Specification Language

d. Verification and Design Model - System Language

14- VDM-SL is primarily used for:

a. Database management

b. Software specification and verification

c. Real-time operating systems

d. Web development

15- In VDM-SL, what does the specification define?

a. The appearance of the user interface

b. The internal workings of algorithms

c. The system's requirements and behavior

d. The system's deployment architecture

16- In set theory, the union of two sets A and B contains:

a. Elements common to both sets

b. Elements only in set A

c. Elements only in set B

d. All elements from both sets

17- What does the cardinality of a set represent?

a. The number of elements in the set

b. The order of elements in the set

c. The sum of elements in the set

d. The product of elements in the set

18- The intersection of two sets A and B contains:

a. Elements common to both sets

b. Elements only in set A

c. Elements only in set B

d. All elements from both sets

19- A sequence is an ordered list of:

a. Random elements

b. Unique elements

c. Indexed elements

d. Unordered elements

20- The length of a sequence is also known as:

a. Width

b. Depth

c. Cardinality

d. Order

21- In the context of sequences, what does indexing refer to?

a. Reordering the elements

b. Assigning a unique position to each element

c. Combining two sequences

d. Grouping elements based on a common attribute

22- What defines composite objects in formal methods?

a. Objects with complex data structures

b. Objects with simple data structures

c. Objects with high-level abstraction

d. Objects with low-level abstraction

23- Composition in composite objects involves:

a. Combining multiple objects to create a new one

b. Splitting an object into multiple parts

c. Changing the properties of an object

d. Deleting elements from an object

24- In formal methods, what is the purpose of abstraction in composite objects?

a. Simplifying complex objects for better understanding

b. Introducing unnecessary details for clarity

c. Increasing the size of composite objects

d. Reducing the efficiency of operations on objects

25- In formal methods, a map is a relation between:

a. Two sets

b. Three sets

c. Multiple sets

d. No sets

26- What is a bijective map?

a. Every element in the domain is paired with exactly one element in the codomain

b. Some elements in the domain have multiple pairs in the codomain

c. Multiple elements in the domain are paired with one element in the codomain

d. No elements are paired between the domain and the codomain

27- In the context of maps, what does the codomain represent?

a. The set of all possible inputs

b. The set of all possible outputs

c. The relationship between inputs and outputs

d. The intersection of input and output sets

28- Z in formal methods is used for:

a. Set theory

b. Specification and verification

c. Concurrent programming

d. Database management

29- In Z notation, what does the schema represent?

a. A set of operations

b. A set of axioms

c. A set of assertions

d. A set of variables and their relationships

30- In Z notation, what is an invariant?

a. A statement that must always be true during the execution of a system

b. A statement that is true only in specific cases

c. A statement that is false during the execution of a system

d. A statement that is not relevant to the system's behavior

31- Well-formedness in formal methods ensures:

a. Efficient program execution

b. Compliance with defined rules or syntax

c. High-level abstraction

d. Low-level abstraction

32- What is the significance of well-formed methods in software development?

a. Improved debugging

b. Enhanced program performance

c. Increased code readability

d. Reduced software maintenance costs

33- Which of the following is a common consequence of violating well-formedness rules?

a. Improved system reliability

b. Code optimization

c. Run-time errors

d. Increased development speed

34- What is the primary advantage of formal methods over informal methods in software engineering?

a. Faster development speed

b. Improved communication among team members

c. Rigorous reasoning for correctness

d. Better aesthetics of the code

35- In the context of formal methods, what is the purpose of model checking?

a. Optimizing code for performance

b. Verifying that a model satisfies specified properties

c. Conducting user acceptance testing

d. Improving software documentation

36- How does a race condition occur in concurrent systems?

a. When multiple processes are synchronized perfectly

b. When multiple processes compete for shared resources

c. When processes are independent of each other

d. When processes run sequentially

37- In set theory, what does the power set of a set A represent?

a. The set of all elements in A

b. The set of all subsets of A, including the empty set

c. The set of all prime numbers in A

d. The set of all even numbers in A

38- What distinguishes a sequence from a set in formal methods?

a. Sequences have an order, while sets do not.

b. Sequences cannot have duplicate elements.

c. Sequences are always finite.

d. Sets can only contain numerical elements.

39- In composite objects, what does encapsulation refer to?

a. Combining multiple objects to form a new one

b. Restricting access to the internal details of an object

c. Breaking an object into its constituent parts

d. Modifying the properties of an object dynamically

40- What is the purpose of a schema in Z notation?

a. Defining a set of variables and their relationships

b. Describing the internal implementation details of a system

c. Enumerating all possible outcomes of a system

d. Specifying the user interface of a software application

More MCQS

1. Which operator in VDM-SL returns the number of elements in a given set?

- A) card
- B) size
- C) count
- D) len
- **Answer: A) card**

2. The elems operator in VDM-SL returns:

- A) The number of elements in a sequence.
- B) A set containing all the members of the sequence.
- C) The first element in the sequence.
- D) The length of the sequence.
- **Answer: B) A set containing all the members of the sequence.**

3. In VDM-SL, the hd operator provides:

- A) The length of the sequence.
- B) The set of all indices of the sequence.
- C) The first element in the sequence.
- D) All but the first element in the sequence.
- **Answer: C) The first element in the sequence.**

4. What does the tl operator in VDM-SL return?

- A) The last element in the sequence.
- B) The length of the sequence.

- C) A set of all indices of the sequence.
- D) A sequence containing all but the first element.
- **Answer: D) A sequence containing all but the first element.**

5. The concatenation operator in VDM-SL is represented by:

- A) &
- B) ^
- C) +
- D) *
- **Answer: B) ^**

6. In VDM-SL, which operator is used to override a sequence?

- A) ^
- B) †
- C) &
- D) +
- **Answer: B) †**

7. The inds operator in VDM-SL returns:

- A) The length of the sequence.
- B) The first element in the sequence.
- C) The set of all indices of the sequence.
- D) The last element in the sequence.
- **Answer: C) The set of all indices of the sequence.**

8. Which set operation in VDM-SL is not commutative?

- A) Union
- B) Intersection
- C) Difference
- D) Symmetric Difference
- **Answer: C) Difference**

9. In VDM-SL, the symbol for set membership is:

- A) ∉
- B) ∈

- C) ⊆
- D) ∪
- **Answer: B) ∈**

10. The LIMIT value in the Patient Register example in VDM-SL is:

- A) 100
- B) 150
- C) 200
- D) 250
- **Answer: C) 200**

11. The Patient type in VDM-SL is defined as:

- A) INTEGER
- B) STRING
- C) TOKEN
- D) BOOLEAN
- **Answer: C) TOKEN**

12. Which type of set operation returns TRUE if all elements in the first set are also in the second set?

- A) Union
- B) Intersection
- C) Subset
- D) Difference
- **Answer: C) Subset**

13. In VDM-SL, which operator is used to remove duplicates from a sequence?

- A) card
- B) len
- C) elems
- D) hd
- **Answer: C) elems**

14. The comprehension environment rule in VDM-SL deals with:

- A) Identifiers

- B) Null set
- C) Power-set
- D) Cartesian product
- **Answer: B) Null set**

15. The well-typing rule in VDM-SL ensures:

- A) Identifiers are used correctly.
- B) The null set is represented accurately.
- C) Tuples are formed correctly.
- D) Terms are typed correctly and values are within their types.
- **Answer: D) Terms are typed correctly and values are within their types.**

16. The schema designator rule in VDM-SL is part of:

- A) Well-typing rules
- B) Environment rules
- C) Tuple rules
- D) Function application rules
- **Answer: A) Well-typing rules**

17. What is a schema in the context of VDM and Z?

- A) A mathematical structure
- B) A sequence operation
- C) A set operation
- D) A type of predicate
- **Answer: A) A mathematical structure**

18. The semantic function for terms in VDM-SL provides:

- A) Type and value of the term
- B) Length of the sequence
- C) Set of all indices
- D) First element in the sequence
- **Answer: A) Type and value of the term**

19. The variety in VDM-SL refers to:

- A) A specific type of set

- B) A class of structures
- C) A schema designator
- D) An operator for sequences
- **Answer: B) A class of structures**

20. The notation for denotational semantics in VDM-SL is based on:

- A) Sequences
- B) Sets
- C) Cartesian products
- D) Tuples
- **Answer: B) Sets**

21. In VDM-SL, a null set is:

- A) A set with no elements
- B) A set with all elements
- C) A set with one element
- D) A set with repeated elements
- **Answer: A) A set with no elements**

22. The well-typing rule for the null set ensures:

- A) The set is empty.
- B) The set has exactly one element.
- C) The set has no repeated elements.
- D) The set contains all elements of another set.
- **Answer: A) The set is empty.**

23. In VDM-SL, a tuple is:

- A) A single value
- B) A sequence of elements
- C) A pair of type and value
- D) A function application
- **Answer: B) A sequence of elements**

24. The selection operator in VDM-SL is used to:

- A) Choose elements from a set

- B) Select elements from a tuple
- C) Apply a function
- D) Create a power-set
- **Answer: B) Select elements from a tuple**

25. The μ-term in VDM-SL is a part of:

- A) Sequence operations
- B) Well-typing rules
- C) Environment rules
- D) Function application rules
- **Answer: D) Function application rules**

26. In VDM-SL, the power-set operation:

- A) Returns a set of all subsets of a given set.
- B) Returns the number of elements in a set.
- C) Returns the first element of a set.
- D) Returns a sequence of elements.
- **Answer: A) Returns a set of all subsets of a given set.**

27. Which VDM-SL operator is used to describe the cartesian product of sets?

- A) ∪
- B) ∩
- C) ×
- D) ⊆
- **Answer: C) ×**

28. The λ-term in VDM-SL represents:

- A) A set operation
- B) A function definition
- C) A tuple selection
- D) A sequence concatenation
- **Answer: B) A function definition**

29. The comprehension operator in VDM-SL is used to:

- A) Define sets by specifying a condition.

- B) Calculate the cardinality of a set.
- C) Return the first element of a set.
- D) Concatenate sequences.
- **Answer: A) Define sets by specifying a condition.**

30. In VDM-SL, the signature defines:

- A) The type of a variable.
- B) The elements of a set.
- C) The length of a sequence.
- D) The operations on a tuple.
- **Answer: A) The type of a variable.**

31. The schema in VDM-SL contains:

- A) Only the variable declarations.
- B) Only the invariants.
- C) Variable declarations and invariants.
- D) Only the function definitions.
- **Answer: C) Variable declarations and invariants.**

32. The well-typing rule in VDM-SL checks:

- A) Only the structure of the terms.
- B) Only the environment rules.
- C) Both the type and the value of terms.
- D) Only the value of terms.
- **Answer: C) Both the type and the value of terms.**

33. The restriction operator in VDM-SL:

- A) Limits the elements of a set.
- B) Combines two sequences.
- C) Defines the cartesian product of sets.
- D) Applies a function to a tuple.
- **Answer: A) Limits the elements of a set.**

34. The projection operator in VDM-SL:

- A) Selects elements from a sequence.

- B) Applies a function to a set.
- C) Defines a subset of elements.
- D) Returns a specific element from a tuple.
- **Answer: D) Returns a specific element from a tuple.**

35. In VDM-SL, the mapping operator:

- A) Returns the length of a sequence.
- B) Maps elements of one set to another set.
- C) Concatenates two sequences.
- D) Selects elements from a tuple.
- **Answer: B) Maps elements of one set to another set.**

36. The selection operator in VDM-SL for sets:

- A) Returns the cardinality of the set.
- B) Selects a subset based on a condition.
- C) Concatenates two sets.
- D) Defines the cartesian product.
- **Answer: B) Selects a subset based on a condition.**

37. The operator in VDM-SL that concatenates two sequences is:

- A) †
- B) &
- C) ^
- D) +
- **Answer: C) ^**

38. In VDM-SL, the well-typed rule ensures:

- A) The elements of a sequence are unique.
- B) The type and value of terms are correct.
- C) The indices of a sequence are valid.
- D) The length of a sequence is within bounds.
- **Answer: B) The type and value of terms are correct.**

Which operator in VDM-SL returns the number of elements in a given set?

- a) len
- b) card **(Correct Answer)**
- c) elems
- d) inds

In VDM-SL, which operator removes duplicate elements from a sequence?

- a) hd
- b) tl
- c) elems **(Correct Answer)**
- d) inds

The 'head' (hd) operator in VDM-SL returns:

- a) The last element in the sequence
- b) The first element in the sequence **(Correct Answer)**
- c) All but the first element
- d) The length of the sequence

In VDM-SL, the 'tail' (tl) operator gives us:

- a) The first element in the sequence
- b) All but the first element **(Correct Answer)**
- c) The length of the sequence
- d) The set of all indices

What does the concatenation operator (^) do in VDM-SL?

- a) Joins two sequences together **(Correct Answer)**
- b) Removes duplicates from a sequence
- c) Returns the length of a sequence
- d) Overrides elements in a sequence

The override operator in VDM-SL is represented by which symbol?

- a) ^
- b) † **(Correct Answer)**
- c) ∈
- d) ∉

Which of the following operations is commutative?

- a) Set difference
- b) Intersection **(Correct Answer)**
- c) Membership
- d) Subset

What is the result of {a, d, e} ⊆ {a, b, c, d, e, f}?

- a) FALSE
- b) TRUE **(Correct Answer)**
- c) NULL
- d) ERROR

In the context of set operations, what does the cardinality operator (card) return?

- a) The union of two sets
- b) The intersection of two sets
- c) The number of elements in a set **(Correct Answer)**
- d) Whether an element belongs to a set

Which symbol represents the membership operator?

- a) ∈ **(Correct Answer)**
- b) ⊆
- c) ∉
- d) ∪

Which of the following is not a type of elementary predicate?

- a) Equality between terms of the same type
- b) Membership between a term and a set
- c) Logical constants true and false
- d) Sequence length **(Correct Answer)**

What does the term 'Schema' refer to in formal methods?

- a) A type of data structure
- b) A logical representation of a structure **(Correct Answer)**
- c) A sequence of operations
- d) An abstract syntax tree

What is the result of the set difference {a, b, c} \ {b}?

- a) {b}
- b) {a, c} **(Correct Answer)**
- c) {a, b, c}
- d) {}

In VDM-SL, the 'inds' operator returns:

- a) The indices of a sequence **(Correct Answer)**
- b) The elements of a sequence
- c) The head of a sequence
- d) The tail of a sequence

Which of the following set operations is not commutative?

- a) Union
- b) Intersection
- c) Set difference **(Correct Answer)**
- d) Cartesian product

Which operation checks whether a particular element is in a set?

- a) Union
- b) Intersection
- c) Membership **(Correct Answer)**
- d) Cardinality

In VDM-SL, what is the type of a term determined by?

- a) Its position in a sequence
- b) Its environment **(Correct Answer)**
- c) The number of elements in the term
- d) The length of the term

What does the 'type' function do in a signature?

- a) Assigns a value to a variable
- b) Assigns a type to a variable **(Correct Answer)**
- c) Defines the structure of a schema
- d) Returns the cardinality of a set

Which of the following is not part of a schema's declaration in VDM-SL?

- a) Given-set names
- b) Variable names
- c) Type of each variable
- d) Sequence length **(Correct Answer)**

What is the significance of the 'axiom part' in a schema?

- a) Defines the type of each variable
- b) Describes relationships among variables **(Correct Answer)**
- c) Specifies the length of sequences
- d) Lists all indices of sequences

In VDM-SL, a 'null set' is represented by which symbol?

- a) ∅ **(Correct Answer)**
- b) ∈
- c) ∉
- d) ∪

Which operator in VDM-SL gives the first element of a sequence?

- a) hd **(Correct Answer)**
- b) tl
- c) elems
- d) inds

Which of the following is a correct representation of a set intersection in VDM-SL?

- a) J ∪ K
- b) J ∩ K **(Correct Answer)**
- c) J \ K
- d) J ∈ K

In formal methods, which property of terms states that the value of a term is always an element of its type?

- a) Well-typing rule
- b) Environment rule
- c) Tuple rule
- d) Property of terms **(Correct Answer)**

What is the result of {a, b, c} ⊆ {a, b, c}?

- a) TRUE **(Correct Answer)**
- b) FALSE
- c) NULL
- d) ERROR

What does the 'len' operator return in VDM-SL?

- a) The length of a sequence **(Correct Answer)**
- b) The first element of a sequence
- c) The set of all indices of a sequence
- d) The set of all elements of a sequence

In VDM-SL, what does the 'power-set' operator do?

- a) Returns the length of a sequence
- b) Returns the first element of a sequence
- c) Returns all subsets of a set **(Correct Answer)**
- d) Overrides elements in a sequence

Which of the following is a commutative operation in set theory?

- a) Union **(Correct Answer)**
- b) Set difference
- c) Sequence concatenation
- d) Membership

In VDM-SL, which operator would you use to join two sequences?

- a) †
- b) ∈
- c) ^
- d) ⊆

Which operator in VDM-SL is used to override elements in a sequence?

- a) ^
- b) ∈
- c) † **(Correct Answer)**
- d) ⊆

What does the 'Cartesian product' operation return in set theory?

- a) A set of all subsets of a set
- b) A set of ordered pairs from two sets **(Correct Answer)**
- c) The union of two sets
- d) The intersection of two sets

In the context of formal methods, what is a 'variety'?

- a) A sequence of operations
- b) A signature with a set of structures **(Correct Answer)**
- c) A type of term
- d) An environment rule

Which of the following operators checks whether two sets are equal?

- a) ∪
- b) ∩
- c) ⊆
- d) ⊆ with TRUE result **(Correct Answer)**

In VDM-SL, the 'comprehension' rule applies to:

- a) Identifiers
- b) Null set
- c) Power-set
- d) Extensive set **(Correct Answer)**

Which operator in VDM-SL removes duplicates and returns all elements of a sequence?

- a) hd
- b) tl
- c) elems **(Correct Answer)**
- d) inds

What is a tautology in propositional logic?

- A. A proposition that is always false
- **B. A proposition that is always true**
- C. A proposition that is sometimes true

- D. A proposition that is undefined

What is the definition of a valid argument?

- **A. An argument where the conclusion is true whenever all premises are true**
- B. An argument that contains only true premises
- C. An argument that contains only false premises
- D. An argument that is sometimes true

What is a fallacy in propositional logic?

- A. A valid argument
- **B. An invalid argument**
- C. An argument with true premises
- D. An argument with a true conclusion

Which logical operator in classical logic assumes all expressions evaluate to TRUE or FALSE?

- A. AND Operator
- B. OR Operator
- **C. IMPLICATION Operator**
- D. Equivalence Operator

What does the symbol ∈ represent in set theory?

- **A. Is an element of**
- B. Is a subset of
- C. Is equal to
- D. Is not an element of

What is a predicate in predicate logic?

- **A. A truth-valued expression containing free variables**
- B. A set of values
- C. A logical operator
- D. A conclusion of an argument

Which operator is used to represent negation in classical logic?

- A. AND Operator
- B. OR Operator
- C. IMPLICATION Operator

- **D. Negation Operator**

What is a proof in formal methods?

- **A. A rigorous demonstration that establishes the correctness of a claim**
- B. A set of premises
- C. A logical operator
- D. A set of propositions

What is the purpose of verification in formal methods?

- **A. To verify that a system meets its desired properties**
- B. To create new logical operators
- C. To generate new sets
- D. To evaluate the truth of a predicate

What is the ZF axiom system in set theory?

- A. A set of logical operators
- **B. A universe of pure sets built from the empty set**
- C. A set of predicates
- D. A type of logical fallacy

What is the union operation in set theory?

- **A. A function from sets to sets that combines elements of sets**
- B. A logical operator
- C. A predicate
- D. A tautology

What does the axiom of infinity ensure in set theory?

- **A. The existence of at least one infinite set**
- B. The existence of finite sets
- C. The existence of empty sets
- D. The existence of tautologies

What is the AND operator used for in classical logic?

- **A. To evaluate conjunctions**
- B. To evaluate disjunctions
- C. To evaluate implications

- D. To evaluate negations

What is the purpose of completeness in formal models?

- **A. To cover all possible scenarios and interactions within the system**
- B. To create new sets
- C. To generate new logical operators
- D. To evaluate the truth of predicates

What does consistency in a formal system ensure?

- **A. The system does not contain conflicting statements**
- B. The system contains only true statements
- C. The system contains only false statements
- D. The system contains undefined statements

What is the purpose of refinement in formal methods?

- **A. To ensure the implementation correctly refines the higher-level specification**
- B. To create new sets
- C. To generate new logical operators
- D. To evaluate the truth of predicates

What does the term 'proof' refer to in formal methods?

- **A. A rigorous demonstration establishing the correctness of a claim**
- B. A set of premises
- C. A logical operator
- D. A set of propositions

What is the purpose of soundness in formal methods?

- **A. To ensure any conclusion drawn from the analysis is guaranteed to be correct**
- B. To create new sets
- C. To generate new logical operators
- D. To evaluate the truth of predicates

What is the purpose of automation in formal methods?

- **A. To reduce the risk of human error and make the verification process more reliable**
- B. To create new sets
- C. To generate new logical operators

- D. To evaluate the truth of predicates

What is the purpose of handling uncertainty in formal methods?

- **A. To provide insights into the behavior of the system under different conditions**
- B. To create new sets
- C. To generate new logical operators
- D. To evaluate the truth of predicates

What is the purpose of the power set axiom in set theory?

- **A. To assert the existence of the power set of any set**
- B. To create new sets
- C. To generate new logical operators
- D. To evaluate the truth of predicates

What is the purpose of the pair-sets axiom in set theory?

- **A. To allow the construction of two-element sets**
- B. To create new sets
- C. To generate new logical operators
- D. To evaluate the truth of predicates

What is the purpose of the null set axiom in set theory?

- **A. To assert the existence of a set with no elements**
- B. To create new sets
- C. To generate new logical operators
- D. To evaluate the truth of predicates

What does the term 'predicate' refer to in predicate logic?

- **A. A truth-valued expression containing free variables**
- B. A set of values
- C. A logical operator
- D. A conclusion of an argument

What is the purpose of equivalence operators in classical logic?

- **A. To evaluate the equivalence of two propositions**
- B. To evaluate conjunctions
- C. To evaluate disjunctions

- D. To evaluate negations

What is the purpose of the OR operator in classical logic?

- **A. To evaluate disjunctions**
- B. To evaluate conjunctions
- C. To evaluate implications
- D. To evaluate negations

What is the purpose of the IMPLICATION operator in classical logic?

- **A. To evaluate implications**
- B. To evaluate conjunctions
- C. To evaluate disjunctions
- D. To evaluate negations

What does the term 'undefined' mean in three-valued logic?

- **A. A proposition that cannot be evaluated as TRUE or FALSE**
- B. A proposition that is always true
- C. A proposition that is always false
- D. A proposition that is sometimes true

What is the purpose of soundness in analysis of formal methods?

- **A. To ensure any conclusion drawn from the analysis is guaranteed to be correct**
- B. To create new sets
- C. To generate new logical operators
- D. To evaluate the truth of predicates

What is the purpose of tool support in formal methods?

- **A. To improve the usability and effectiveness of the method**
- B. To create new sets
- C. To generate new logical operators
- D. To evaluate the truth of predicates

Which of the following is a tautology?

- A) $p \vee \neg p$
- B) $p \wedge \neg p$
- C) $p \rightarrow p$

- D) $p \leftrightarrow p$
- **Correct Answer: A) $p \vee \neg p$**

Which of the following propositions is a contradiction?

- A) $p \vee q$
- B) $p \wedge \neg p$
- C) $p \rightarrow q$
- D) $p \leftrightarrow q$
- **Correct Answer: B) $p \wedge \neg p$**

What is the definition of an argument in propositional logic?

- A) A proposition that is always true.
- B) A proposition that is always false.
- C) An assertion that a set of propositions yields another proposition.
- D) A method to prove the consistency of a proposition.
- **Correct Answer: C) An assertion that a set of propositions yields another proposition.**

What is an argument that is not valid called?

- A) Tautology
- B) Contradiction
- C) Fallacy
- D) Proposition
- **Correct Answer: C) Fallacy**

Which logic system assumes all expressions evaluate to TRUE or FALSE?

- A) Predicate Logic
- B) Three-Valued Logic
- C) Classical Logic
- D) Modal Logic
- **Correct Answer: C) Classical Logic**

In Three-Valued Logic, what are the possible values of a proposition?

- A) TRUE, FALSE, MAYBE
- B) TRUE, FALSE, UNKNOWN
- C) TRUE, FALSE, UNDEFINED

- D) TRUE, FALSE, NULL
- **Correct Answer: C) TRUE, FALSE, UNDEFINED**

What does the symbol ∈ represent in set theory?

- A) Subset of
- B) Is an element of
- C) Union
- D) Intersection
- **Correct Answer: B) Is an element of**

Which quantifier expresses that a predicate is true for all elements in a set?

- A) Existential Quantifier
- B) Universal Quantifier
- C) Unique Existential Quantifier
- D) Null Quantifier
- **Correct Answer: B) Universal Quantifier**

Which quantifier indicates that at least one element in the set meets the criterion?

- A) Existential Quantifier
- B) Universal Quantifier
- C) Unique Existential Quantifier
- D) Null Quantifier
- **Correct Answer: A) Existential Quantifier**

What is a proof in formal methods?

- A) A demonstration that a system is incomplete.
- B) A rigorous argument establishing the correctness of a claim.
- C) An informal method to argue about system properties.
- D) A way to show that a proposition is always false.
- **Correct Answer: B) A rigorous argument establishing the correctness of a claim.**

Which of the following is NOT a purpose of proofs in formal methods?

- A) Verification
- B) Correctness
- C) Refinement

- D) Inconsistency
- **Correct Answer: D) Inconsistency**

What does consistency in a formal system ensure?

- A) The system is complete.
- B) The system does not contain conflicting statements.
- C) The system is always true.
- D) The system is always false.
- **Correct Answer: B) The system does not contain conflicting statements.**

Which operator in Three-Valued Logic returns FALSE if both operands are FALSE, otherwise returns TRUE?

- A) AND
- B) OR
- C) IMPLICATION
- D) NEGATION
- **Correct Answer: B) OR**

In VDM-SL, what is the maximum number of patients that can be registered in the Patient Register?

- A) 100
- B) 150
- C) 200
- D) 250
- **Correct Answer: C) 200**

Which predicate indicates that a variable x studies a subject y?

- A) C(x)
- B) Prime(n)
- C) Studies(x, y)
- D) M(x)
- **Correct Answer: C) Studies(x, y)**

What does the VDM-SL keyword 'inv' stand for in the context of data types?

- A) Invocation
- B) Invariant

- C) Inverse
- D) Invalid
- **Correct Answer: B) Invariant**

Which term is used to describe a formal method that supports a rich set of constructs and language features?

- A) Consistency
- B) Completeness
- C) Expressiveness
- D) Soundness
- **Correct Answer: C) Expressiveness**

In the context of proofs, what does verification ensure?

- A) That a system is incomplete.
- B) That a system meets its desired properties or requirements.
- C) That a system is always false.
- D) That a system contains inconsistencies.
- **Correct Answer: B) That a system meets its desired properties or requirements.**

Which operator in Three-Valued Logic returns UNDEFINED if the operand is UNDEFINED?

- A) AND
- B) OR
- C) IMPLICATION
- D) NEGATION
- **Correct Answer: D) NEGATION**

In predicate logic, what does the expression ∀x ∈ Cats ● M(x) mean?

- A) There exists an x which is a member of Cats.
- B) All cats chase mice.
- C) No cat chases mice.
- D) Some cats chase mice.
- **Correct Answer: B) All cats chase mice.**

Subjective Questions (3 Marks)

Key objectives of formal methods in software engineering:

- **Formal methods aim to provide rigorous, mathematical techniques for analyzing, specifying, and verifying software systems to ensure reliability, correctness, and robustness. Unlike informal methods, which use natural language, formal methods use precise, mathematical models to eliminate ambiguities and ensure consistency(CS636 final term week(7…).**

Role of mathematical models in software specification and verification:

- **Mathematical models provide formal representations of system behavior, enabling precise and unambiguous specifications. They allow for rigorous analysis and proof of correctness, ensuring the system meets its specifications, detects errors early, and enhances reliability (CS636 final term week(7…).**

Concept of well-formedness in concurrent systems:

- **Well-formedness ensures that models or specifications adhere to syntactic rules and constraints, allowing formal analysis to check for consistency, correctness, safety, and liveness. Adhering to well-formedness principles ensures reliable and predictable behavior in concurrent software(CS636 final term week(7…).**

Differences between propositional and predicate logic:

- **Propositional logic deals with simple true/false statements and is useful for basic logical relationships. Predicate logic extends this by handling predicates with variables and quantifiers, making it suitable for more complex specifications involving relationships between objects(CS636 final term week(7…).**

Key features and purposes of VDM-SL:

- **VDM-SL is a formal specification language for modeling software systems. It provides precise, unambiguous descriptions using mathematical notation, supports modularity, and enables rigorous analysis and verification of system properties(CS636 final term week(7…).**

Definition and use of sets in formal methods:

- **Sets are well-defined collections of objects used to represent groups of related elements in software specification and design. They help in providing clear and concise descriptions of system behaviors and constraints, such as valid inputs or system states**

Subjective Questions (5 Marks)

1- Explain the process of using mathematical models for software specification and verification. Discuss the advantages and challenges associated with this approach,

providing examples of real-world applications where mathematical models have proven beneficial.

1. Using Mathematical Models for Software Specification and Verification

Mathematical models in software specification and verification involve formal methods to define and check system properties. These methods utilize mathematical logic to create specifications that are both precise and unambiguous. The advantages of this approach include increased reliability, the ability to rigorously verify correctness, and improved consistency in complex systems. Challenges include the steep learning curve, the need for specialized knowledge, and the potential for increased development time.

Examples of real-world applications:

- **Aviation software**: Ensures safety and correctness in flight control systems.
- **Cryptographic protocols**: Verifies security properties in communication systems.

2- In the context of concurrent systems, elaborate on the concept of well-formedness. Discuss how the principles of well-formedness contribute to the design and implementation of concurrent software systems. Provide examples and explore potential pitfalls in neglecting well-formedness.

2 Well-Formedness in Concurrent Systems

Well-formedness refers to adherence to syntactic rules and constraints of a formal language, ensuring structural correctness. In concurrent systems, well-formedness ensures that specifications are free from ambiguities and errors, which is critical for applying formal analysis techniques to verify properties such as consistency, correctness, and safety. Neglecting well-formedness can lead to misinterpretations and incorrect results.

Example:

- **Operating systems**: Ensuring that concurrent processes adhere to defined rules prevents race conditions and deadlocks

3- Compare and contrast the concepts of sets and sequences in formal methods. Illustrate how these mathematical structures are employed in software specification. Provide examples to demonstrate how sets and sequences contribute to the clarity and precision of specifications.

3 Sets vs. Sequences in Formal Methods

Sets:

- Unordered collections of unique elements.
- Used for modeling collections where order is irrelevant.
- Example: A set of registered users in a system.

Sequences:

- Ordered collections, allowing duplicates.

- Used when the order of elements is significant.
- Example: A sequence of user actions in a session.

Both sets and sequences enhance clarity and precision in software specifications by providing well-defined structures to model different types of data collections.

Usage in software specification:

- Sets model unique, unordered items (e.g., user IDs).
- Sequences model ordered events or steps

4- Explore the role of maps and Z notation in formal methods. Discuss how maps are used to establish relationships between sets and how Z notation facilitates specification and verification. Provide a detailed explanation of how Z notation enhances the expressiveness and precision of formal specifications, highlighting its application in real-world scenarios.

4 Maps and Z Notation in Formal Methods

Maps:

- Establish relationships between sets.
- Useful for defining functions and mappings in specifications.

Z Notation:

- A formal specification language based on set theory and predicate logic.
- Enhances expressiveness and precision in formal specifications.

Application:

- **Z notation** helps in the formal specification of software systems by providing a structured way to describe system states and operations. It is used in various domains such as railway signaling systems and financial software to ensure correctness and reliability.

Benefits:

- Improved expressiveness and precision.
- Facilitates rigorous verification of system properties.

Example:

- Z notation can specify the behavior of a banking system, detailing account operations and ensuring consistency and correctness in transaction processing

4:07 PM

- The union of two sets, j and k returns a set that contains all the elements of the set j and all the elements of the set k.

```
J ∪ K   (J Union K)
```

- Example:

```
if      j = {<MON>, <TUE>, <WED>, <SUN>}
and     k = {<MON>, <FRI>, <TUE>}
then    j ∪ k = {<MON>, <TUE>, <WED>, <SUN>, <FRI>}
```

126

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Set Operations: Intersection

- The intersection of two sets j and k returns a set that contains all the elements that are common to both j and

```
J ∩ K   (J intersection K)
```

- Example:

```
if      j = {<MON>, <TUE>, <WED>, <SUN>}
and     k = {<MON>, <FRI>, <TUE>}
then    j ∩ k = {<MON>, <TUE>}
```

127

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Set Operations: Difference

- The difference of j and k is the set that contains all the elements that belong to j but do not belong to k.

```
J \ K   (J difference K)
```

- Example:

```
if      j = {<MON>, <TUE>, <WED>, <SUN>}
and     k = {<MON>, <FRI>, <TUE>}
then    j \ k = {<WED>, <SUN>}
```

```
Incorrect: {<MON>, <TUE>, <WED>} \ <TUE>      Correct: {<MON>, <TUE>, <WED>} \ {<TUE>}
```

128

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Set Operations: Equal

Edit Annotate Fill & Sign Convert All

4:07 PM
1.7 K/s
50

- It returns TRUE if all the elements in the first set are also elements of the second set and FALSE otherwise
- Example:
  - $\{a, d, e\} \subseteq \{a, b, c, d, e, f\}$ ... TRUE
  - $\{a, b, c, d, e, f\} \subseteq \{a, d, e\}$ ... FALSE
- Returns TRUE if both sets are equal
  - $\{a, d, e\} \subseteq \{d, a, e\}$ ... TRUE

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

et Operations: Cardinality

- The cardinality operator of VDM-SL (card) returns the number of elements in a given set.

```
card {7, 2, 12} = 3
card {4,...,10} = 7
card { } = 0
```

- As repetition is not significant in sets, repeated elements are counted only once when calculating the cardinality

```
card {7, 2, 12, 2, 2} =card {7, 2, 12} = 3
```

ference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

CS636 Formal Methods

Topic 10.3

Topics Covered

Patient Register Class

Patient Surgery Clinic

Edit Annotate Fill & Sign Convert All

Sequences

- As sequence is an ordered collection, so:
  - [a, d, f] ≠ [a, f, d]
- Empty Sequence is expressed as:
  - [ ]
- The elements of a sequence are numbered, starting from 1, from left to right. We can refer to a particular element of a sequence by placing the position of the element in brackets
- s(3)=f queue(4) = WINSTON

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Sequence Operators

s = [a, d, f, a, d, d, c]
queue = [MICHAEL, VARINDER, ELIZABETH, WINSTON, JUDITH]

- The len operator gives us the length of the sequence.
  - *len s = 7*
  - *len queue = 5*
- The elems operator returns a set that contains all the members of the sequence (removes the duplicates):
  - *elems s = {a, d, f, c}*
  - *elems queue = {MICHAEL, VARINDER, ELIZABETH, WINSTON, JUDITH}*

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Sequence Operators

s = [a, d, f, a, d, d, c]
queue = [MICHAEL, VARINDER, ELIZABETH, WINSTON, JUDITH]

- The head (hd) operator gives us the first element in the sequence
- The tail (tl) operator gives us a sequence containing all but the first element
  - *hd s = s(1) = a*
  - *tl s = [d, f, a, d, d, c]*
  - *hd queue = MICHAEL*
  - *tl queue = [VARINDER, ELIZABETH, WINSTON, JUDITH]*

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Sequence Operators

- The concatenation operator (^) operates on two sequences, and returns a sequence that consists of the two sequences joined together

if first = [w, e, r, w]
and second = [t, w, q]
then
first^second = [w, e, r, w, t, w, q]

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

4:04 PM

thisTime = μ(someTime, minute ↦ 0, second ↦ 0)

thisTime = mk-Time(someTime.hour,0,0)

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

CS636 Formal Methods

12.2

Topics Covered

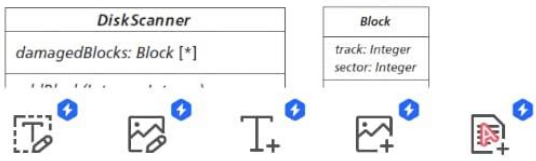
DiskScanner System

DiskScanner System

- **software designed to keep track of damaged blocks on the surface of a disk.**
- ***A disk is divided into a number of tracks***
- ***each track into a number of sectors.***
- ***block = track + sector number***
- ***DiskScanner class***

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

UML representation



Edit Annotate Fill & Sign Convert All

4:03 PM 335 B/s 61



$m = \{a \mapsto y, b \mapsto x, C \mapsto x, d \mapsto z\}$

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Maps

- **By definition, all the domain elements in a map are unique.**
- **The ordering of the maplets is not significant – the map m above could be specified, without changing the meaning, as:**

$m = \{a \mapsto y, b \mapsto x, c \mapsto x, d \mapsto z\}$

$m = \{d \mapsto z, a \mapsto y, c \mapsto x, b \mapsto x\}$

Empty map $\{\mapsto\}$

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Map Operators

- **Domain and Range Operator**

The *domain* operator, **dom**, returns the set of all the domain elements of the maplets.

The *range* operator, **rng**, returns the set of all the range elements.

$m1 = \{a \mapsto 1, b \mapsto 2, c \mapsto 2, d \mapsto 3, e \mapsto 4\}$

$m2 = \{a \mapsto 2, f \mapsto 1, c \mapsto 7\}$

$m3 = \{f \mapsto 2, g \mapsto 6\}$

- **dom** $m1 = \{a, b, c, d, e\}$
- **rng** $m1 = \{1, 2, 3, 4\}$
- **dom** $m2 = \{a, f, c\}$
- **rng** $m2 = \{1, 2, 7\}$

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Map Operators

- **Union Operator**

$m1 = \{a \mapsto 1, b \mapsto 2, c \mapsto 2, d \mapsto 3, e \mapsto 4\}$

$m2 = \{a \mapsto 2, f \mapsto 1, c \mapsto 7\}$

$m3 = \{f \mapsto 2, g \mapsto 6\}$

$m1 \cup m3 = \{a \mapsto 1, b \mapsto 2, c \mapsto 2, d \mapsto 3, e \mapsto 4, f \mapsto 2, g \mapsto 6\}$

- **The union operator is defined only if no two domain**



Edit Annotate Fill & Sign Convert All

4:03 PM
20.4 K/s
61
50
Edit
Annotate
Fill & Sign
Convert
All

238

Formal Methods

Topic#128
Introduction to Z

239

Z-Language

- The Z notation is a language and a style for expressing formal specifications of computing systems.
- It is based on a typed set theory, and the notion of a "schema" is one of its key features.
- A schema consists of a collection of named objects with a relationship specified by some axioms, and Z provides notations for defining schemas and later combining them in various ways, so that large specifications can be built up in stages.
- Schemas can have generic parameters, and there are operations in Z for creating instances of generic schemas.

240

Use of Schemas

- The use of schemas allows the specification to be presented gradually, with a close correspondence between the mathematical text and prose commentary. This makes it easy to explain the mathematics as it is presented and to relate the variables in the mathematical text to the system they describe.

4:01 PM
674 B/s
50

Topics Covered

Proofs

Proof

- A "proof" refers to a rigorous demonstration or argument that establishes the correctness or validity of a particular claim or statement within a formal system.
- It involves applying formal rules of logic and reasoning to show that a property, specification, or assertion holds true for a given model or system under consideration.

Proof

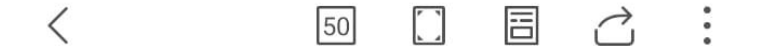
- Proofs play a central role in formal methods for various purposes, including:
  - Verification: Proofs are used to verify that a system meets its desired properties or requirements. For example, in software verification, a proof can demonstrate that a program satisfies a specified safety property, meaning it will never exhibit certain undesirable behaviors.
  - Correctness: Proofs are employed to demonstrate the correctness of algorithms, protocols, or systems. By providing a mathematical guarantee of correctness, formal methods can offer strong assurances of system reliability.
  - Refinement: Proofs are used in the process of refining abstract models to more concrete designs. These proofs ensure that the implementation correctly refines the higher-level specification.

Proof

- Proofs play a central role in formal methods for various purposes, including:
  - Consistency: Proofs can be used to check the consistency of a formal model. A consistent model does not contain conflicting statements or logical contradictions.

Edit Annotate Fill & Sign Convert All

4:00 PM 681 B/s 61



- Now, some hiding operations, the projection operator '|' and the operator '\', which allows particular variables to be hidden, is defined in terms of the function *hide*:

$$
\begin{array}{|l}
hide : VARIETY \times \mathbb{P}\, NAME \rightarrow VARIETY \\
hide_sig : SIG \times \mathbb{P}\, NAME \rightarrow SIG \\
\hline
hide(\theta\, VARIETY, S) = \\
\quad \mu\, VARIETY' \mid \\
\qquad sig' = hide_sig(sig, S) \land \\
\qquad models' = restrict\, sig'\, \{models\} \\
hide_sig(\theta SIG, S) = \\
\quad \mu\, SIG' \mid \\
\qquad sig'.given = sig.given \land \\
\qquad sig'.vars = sig.vars \setminus S \land \\
\qquad sig'.type = sig'.vars \lhd sig.type.
\end{array}
$$

- The universal quantifier likewise corresponds to removing the variables of a schema from the signature of a schema expression and universally quantifying them in the axiom part.
- The classical equivalance between the universal and existential quantifiers continues to hold in the schema calculus.

$$\forall s \bullet se \mathrel{\widehat=} \neg (\exists s \bullet \neg se),$$

where the quantifiers and negation signs have their interpretation as schema operations.

- However, we give here an explicit definition of the universal quantifier in terms of a function *univ*:

$$
\begin{array}{|l}
univ : VARIETY \times VARIETY \times \mathbb{P}\, NAME \nrightarrow VARIETY \\
\hline
univ(\theta\, VARIETY, S) \mathrel{\widehat=} \\
\quad \mu\, VARIETY' \mid \\
\qquad sig' = hide_sig(sig, S) \land
\end{array}
$$



Edit Annotate Fill & Sign Convert All

3:59 PM

Predicates

- Elementary predicates are of three kinds:

-> equality between terms of the same type

-> the membership predicate between a term of type a and another of type P a

-> the logical constants **true** and **false**

- These may be combined with all the usual logical connectives, and qualified with 'for all' and 'there exists':

$$
\begin{aligned}
PRED ::= \; & TERM = TERM \\
| \; & TERM \in TERM \\
| \; & \textbf{true} \\
| \; & \textbf{false} \\
| \; & \neg PRED \\
| \; & PRED \wedge PRED \\
| \; & PRED \vee PRED \\
| \; & PRED \Rightarrow PRED \\
| \; & \exists SCHEMA \bullet PRED \\
| \; & \forall SCHEMA \bullet PRED.
\end{aligned}
$$

- The other kinds of predicates--infix relation symbols, unique quantifiers, and so on--can be defined as syntactic sugar for combinations of these few forms.
- Of course, these forms are not themselves independent: for example,

'$\Rightarrow$' can be defined in terms of '$\neg$ ' and '$\vee$':

$$P \Rightarrow Q \triangleq \neg P \vee Q.$$

A predicate is modelled in the semantics by the set of structures which satisfy it. For the moment, we deal with partially defined terms by fixing a semantics for the equality sign which makes

$$t_1 = t_2$$

true if t1 and t2 are both defined with equal values, and false otherwise, and fixing a similar semantics for the membership sign.

- The rest of the semantics of predicates is a completely classical truth-defintion.

Edit Annotate Fill & Sign Convert All

3:58 PM 647 B/s 61

- A name is an identifier tagged with a level number. The outermost part of a specification is level 0, the variables of a schema are at level 1, and so on: higher numbers are needed e.g. for quantifiers.
- The operation *Tag* of tagging is a bijection because each name determines uniquely the identifier and level number. For convenience, a curried variant *tag* of the tagging function is introduced also.

$$[IDENT, NAME]$$

$$LEVEL \triangleq \mathbb{N}$$

$$Tag : LEVEL \times IDENT \rightarrowtail NAME$$
$$tag : LEVEL \rightarrow IDENT \rightarrow NAME$$
$$tag\ k\ x = Tag(k, x).$$

Rules to determine the name

- Two rules are used to determine the name to which any applied occurrence of an identifier refers:

1. Names at higher levels hide those at lower levels.
2. Variables hide given-set names at the same level.

Environments

- The global given-set names and variables are combined to make a global signature, and a class of models for this signature allows the specified relationship among these names to be recorded. This gives a variety *global*, which is made part of the environment.
- There is also dictionary *sdict* mapping each previously defined schema name to its definition, an object of type SMEANING:

$$ENV$$
$$global : VARIETY$$
$$sdict : WORD \twoheadrightarrow SMEANING$$
$$\forall sm : \mathrm{ran}\ sdict \bullet basis(sm.local.sig)\ \mathrm{subsig}\ global.sig$$

Edit Annotate Fill & Sign Convert All

Implicit Function

- In this method, we use a pre- and postcondition in the same way as we described for operations – a function, of course, does not access the state variables. Here is the add function defined implicitly.

$add(x:\mathbb{R}, y:\mathbb{R})\ z:\mathbb{R}$	
pre	TRUE
post	$z = x + y$

Explicit Function

- The style of this specification is algorithmic, and we explicitly define the method of transforming the inputs to the output. This is illustrated in the following very simple function that adds two numbers together:

$$add: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$$
$$add(x, y) \triangleq x + y$$

- The first line is called the **function signature**.
- The second part is the **function definition.**

Quantification

- The universal quantifier ∀:
- This quantifier enables a predicate to make a statement about all the elements in a particular set. For example, if M(x) is the predicate x chases mice, we could write:
- ∀x ∈ Cats • M(x)
- This reads *For all the x which are members of the set Cats, x chases mice*, or, more simply, *All cats chase mice*.

The nil Value

- It is common in the programming world for a value to be undefined. VDM-SL allows for this concept by including the possibility of a term or expression having the value **nil**, meaning that it is undefined.
- Of course, if we want to allow for this possibility, then we need to slightly modify the type of the variable. We do that by placing square brackets around the type name – for example [N] or [Z] – meaning that a variable of that type can take the value of nil.
- Effectively we are extending the type to include the **nil** value.

inRange Function

- The purpose of this function is to check whether an integer value, val, is within the range MIN and MAX as defined earlier.
- You can see that the use of the equivalence connective ensures that the output is true if the input is in range, but is otherwise false.

```
inRange(val: Z) result: B
pre    true
post   result ⇔ MIN ≤ val ≤ MAX
```

Including Comments

- As with program code, the readability of a VDM-SL specification is greatly enhanced by the inclusion of comments. This is done by introducing the comment with the symbol – –. A new line ends the comment.

```
types
Signal = <INCREASE>|<DECREASE>|<DO_NOTHING>

values
  MAX : Z = 10
  MIN : Z = -10

state IncubatorController of
  requestedTemp : [Z]
  actualTemp : [Z]
  --- both requested and actual temperatures must be in range or equal to nil
  inv mk-IncubatorController (r, a) ≜ (inRange(r) ∨ r = nil) ∧ (inRange(a) ∨
                                                              a = nil)
  --- both requested and actual temperatures are undefined when the system is initialised
  init mk-IncubatorController (r, a) ≜ r = nil ∧ a = nil
end
```

Predicate Logic

- One of the limitations with the propositional logic is that, while it allows us to argue about individual values, it does not give us the ability to argue about sets of values.
- A set is any well-defined, unordered, collection of objects.
  - The set of whole numbers from 1 to 10
  - The set of the days of the week
- In mathematics, we often represent a set elements by lower-case letters. For example:
- A = {s, d, f, h, k}
- B = {a, b, c, d, e, f}
- The symbol $\in$ means 'is an element of'. Therefore the statement 'd is an element of A' is written: $d \in A$

Predicate Logic

- For the purpose of reasoning about sets of values, a more powerful tool than the propositional logic has been devised, namely the predicate logic.
- A predicate is a truth-valued expression containing free variables. These allow the expression to be evaluated by giving different values to the variables.
- Once the variables are evaluated they are said to be bound.

Map Operators

Domain Restriction (⩤) Operator:

- **Defined with two operands. The first is a set and the second is a map.**
- **The result yields a map that contains only those maplets whose domain element is in the set. For example**

$m1 = \{a \mapsto 1, b \mapsto 2, c \mapsto 2, d \mapsto 3, e \mapsto 4\}$
$m2 = \{a \mapsto 2, f \mapsto 1, c \mapsto 7\}$
$m3 = \{f \mapsto 2, g \mapsto 6\}$
$\{a, c, e\} \lhd m1 = \{a \mapsto 1, c \mapsto 2, e \mapsto 4\}$
$\{e, f\} \lhd m2 = \{f \mapsto 1\}$
$\{\} \lhd m3 = \{\mapsto\}$

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Map Operators

Domain Deletion (⩥) Operator:

- **Behaves similar to Domain Restriction operator but it deletes the maplet in questions.**

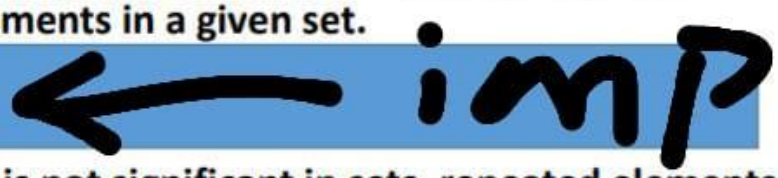
$m1 = \{a \mapsto 1, b \mapsto 2, c \mapsto 2, d \mapsto 3, e \mapsto 4\}$
$m2 = \{a \mapsto 2, f \mapsto 1, c \mapsto 7\}$
$m3 = \{f \mapsto 2, g \mapsto 6\}$
$\{a, c, e\} \rhd m1 = \{b \mapsto 2, d \mapsto 3\}$
$\{e, f\} \rhd m2 = \{a \mapsto 2, c \mapsto 7\}$
$\{\} \rhd m3 = \{f \mapsto 2, g \mapsto 6\}$

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Set Operations: Cardinality

- The cardinality operator of VDM-SL (card) returns the number of elements in a given set.

```
card {7, 2, 12} = 3
card {4,...,10} = 7
card { } = 0
```



- As repetition is not significant in sets, repeated elements are counted only once when calculating the cardinality

```
card {7, 2, 12, 2, 2} =card {7, 2, 12} = 3
```

Set Operations

- ***Three operations that take two sets as input and return a new set as output***
  - ***Set union***
  - ***Set intersection***
  - ***Set difference***
- ***In each case, the types of elements in each set are assumed to be the same.***

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Set Operations: Union

- **The union of two sets, j and k returns a set that contains all the elements of the set j and all the elements of the set k.**

J ∪ K (J Union K)

- **Example:**

if	j = {<MON>, <TUE>, <WED>, <SUN>}
and	k = {<MON>, <FRI>, <TUE>}
then	$j \cup k$ = {<MON>, <TUE>, <WED>, <SUN>, <FRI>}

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Set Operations: Intersection

- **The intersection of two sets j and k returns a set that contains all the elements that are common to both j and k.**

J ∩ K (J intersection K)

- **Example:**

if	j = {<MON>, <TUE>, <WED>, <SUN>}
and	k = {<MON>, <FRI>, <TUE>}
then	$j \cap k$ = {<MON>, <TUE>}



Signatures, structures and varieties

- The declartaive information in a schema is captured in its signature: this records the names of the schema's components or local variables, their types, and the given-set names assumed by the schema.
- **Signatures** are the finite objects, and are thus suitable for mechanical representations and manipulation.
- But a schema contains more information than just the declarations; the axiom part of the schema can describe a relationship among the variables , and this information can be captured by describing which **structures**--assignment of values to the variables--satisfy the axiom part.
- A signature together with the class of appropriately shaped structures is called **variety**.

Sequence Operators

s = [a, d, f, a, d, d, c]

queue = [MICHAEL, VARINDER, ELIZABETH, WINSTON, JUDITH]

- **The len operator gives us the length of the sequence.**
  - ***len s = 7***
  - ***len queue = 5***
- **The elems operator returns a set that contains all the members of the sequence (removes the duplicates):**
  - ***elems s = {a, d, f, c}***
  - ***elems queue = {MICHAEL, VARINDER, ELIZABETH, WINSTON, JUDITH}***

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Sequence Operators

s = [a, d, f, a, d, d, c]

queue = [MICHAEL, VARINDER, ELIZABETH, WINSTON, JUDITH]

- **The head (hd) operator gives us the first element in the sequence;**
- **The tail (tl) operator gives us a sequence containing all but the first element**
  - ***hd s = s(1) = a***
  - ***tl s = [d, f, a, d, d, c]***
  - ***hd queue = MICHAEL***
  - ***tl queue = [VARINDER, ELIZABETH, WINSTON, JUDITH]***

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Specifying a Function in VDM-SL

There are two ways in which we can specify a function in VDM-SL:

- Explicitly
- Implicitly

Specifications

- The smallest specification is a definition, which may introduce some global given-set names, introduce some global variables with some axioms, or define a new schema.
- Definitions may joined with in, and then the names introduced by the first are added to the environment before the second is considered.

$$
\begin{aligned}
SPEC ::= & \ \textbf{given}\ IDENT, \ldots, IDENT \\
| & \ \textbf{let}\ SCHEMA\ \textbf{end} \\
| & \ \textbf{let}\ WORD[IDENT, \ldots, IDENT] = SEXP \\
| & \ SPEC\ \textbf{in}\ SPEC.
\end{aligned}
$$

- A specification is evaluated in an initial environment--this might contain the definitions of the standard mathematical tool-kit, and it produces a new environment, enriched with the new objects defined in the specification.

$$
\begin{array}{|l}
spec : ENV \to SPEC \nrightarrow ENV \\
\hline
spec\ \rho\ [\![\textbf{given}\ z_1, \ldots, z_n]\!] \cong \\
\quad enrich(\rho, new_givens(\rho.global, tag\ 0\ \{\{z_1, \ldots, z_n\}\})) \\
spec\ \rho\ [\![\textbf{let}\ s\ \textbf{end}]\!] \cong enrich(\rho, schema\ \rho\ 0\ [\![s]\!]) \\
spec\ \rho\ [\![\textbf{let}\ w[z_1, \ldots, z_n] = se]\!] \cong \\
\quad \mu\ \rho_1 : ENV \mid \rho_1 \cong \\
\qquad enrich(\rho, new_givens(\rho.global, tag\ 1\ \{\{z_1, \ldots, z_n\}\})) \bullet \\
\quad \mu\ sm : SMEANING \mid \\
\qquad sm.local \cong sexp\ \rho_1\ 1\ [\![se]\!] \wedge \\
\qquad sm.fparam = \langle z_1, \ldots, z_n \rangle \bullet \\
\quad add_schema(\rho, w, sm) \\
spec\ \rho\ [\![z_1\ \textbf{in}\ z_2]\!] \cong \mu\ \rho_1 : ENV \mid \rho_1 \cong spec\ \rho\ [\![z_1]\!] \bullet spec\ \rho_1\ [\![z_2]\!].
\end{array}
$$

- The declaration of global given-set names or global variables enriches the global variety of the environment.
- Because of the semantics of declarations and the definition of *new-givens*, it is meaningless to declare the same global variable or given-set name twice.
- A schema definition adds a new schema name to the dictiona
- The formal generic parameters may be used in the defining s expression, and their order is preserved in the *fparam* seque matching with actual parameters in schema designators.

Sequence Operators

- **The concatenation operator (^) operates on two sequences, and returns a sequence that consists of the two sequences joined together**

if first = [w, e, r, w]
and second = [t, w, q]
then
first^second = [w, e, r, w, t, w, q]

Set Operations

- ***Three operations that take two sets as input and return a new set as output***
  - ***Set union***
  - ***Set intersection***
  - ***Set difference***
- ***In each case, the types of elements in each set are assumed to be the same.***

S.Q

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Set Operations: Union

- **The union of two sets, j and k returns a set that contains all the elements of the set j and all the elements of the set k.**

J ∪ K (J Union K)

- **Example:**

if	j = {<MON>, <TUE>, <WED>, <SUN>}
and	k = {<MON>, <FRI>, <TUE>}
then	$j \cup k$ = {<MON>, <TUE>, <WED>, <SUN>, <FRI>}

Reference: Formal Software Development From VDM to Java by Quentin Charatan and Aaron Kans

Set Operations: Intersection

- **The intersection of two sets j and k returns a set that contains all the elements that are common to both j and k.**

J ∩ K (J intersection K)

- **Example:**

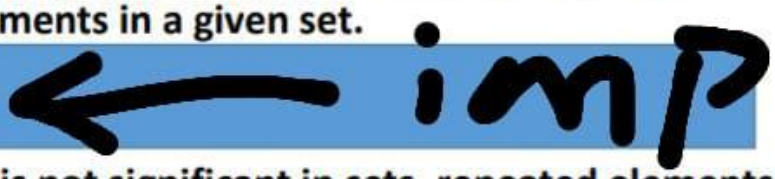
if	j = {<MON>, <TUE>, <WED>, <SUN>}
and	k = {<MON>, <FRI>, <TUE>}
then	$j \cap k$ = {<MON>, <TUE>}



Set Operations: Cardinality

- The cardinality operator of VDM-SL (card) returns the number of elements in a given set.

```
card {7, 2, 12} = 3
card {4,...,10} = 7
card { } = 0
```



- As repetition is not significant in sets, repeated elements are counted only once when calculating the cardinality

```
card {7, 2, 12, 2, 2} =card {7, 2, 12} = 3
```

3:58 PM 647 B/s 61

- A name is an identifier tagged with a level number. The outermost part of a specification is level 0, the variables of a schema are at level 1, and so on: higher numbers are needed e.g. for quantifiers.
- The operation *Tag* of tagging is a bijection because each name determines uniquely the identifier and level number. For convenience, a curried variant *tag* of the tagging function is introduced also.

$$[IDENT, NAME]$$

$$LEVEL == \mathbb{N}$$

$$Tag : LEVEL \times IDENT \rightarrowtail NAME$$
$$tag : LEVEL \rightarrow IDENT \rightarrowtail NAME$$
$$\forall\, tag\ k\ x = Tag(k, x).$$

Rules to determine the name

- Two rules are used to determine the name to which any applied occurrence of an identifier refers:

1. Names at higher levels hide those at lower levels.
2. Variables hide given-set names at the same level.

S.Q

Environments

- The global given-set names and variables are combined to make a global signature, and a class of models for this signature allows the specified relationship among these names to be recorded. This gives a variety *global*, which is made part of the environment.
- There is also dictionary *sdict* mapping each previously defined schema name to its definition, an object of type SMEANING:

$$ENV$$
$$global : VARIETY$$
$$sdict : WORD \pfun SMEANING$$
$$\forall\, sm : ran\ sdict \bullet basis(sm.local_sig)\ subsig\ global_sig$$

Edit Annotate Fill & Sign Convert All

Explicit Function

- The style of this specification is algorithmic, and we explicitly define the method of transforming the inputs to the output. This is illustrated in the following very simple function that adds two numbers together:

$$add: \mathbb{R} \times \mathbb{R} \to \mathbb{R}$$
$$add(x, y) \triangleq x + y$$

- The first line is called the **function signature**.
- The second part is the **function definition.**

Z and other methods

- Styles are divided into model-oriented methods
- Aim of a specification is to construct an abstract model of the information system being specified.
- And property-oriented or algebaric methods, where the aim is to describe a system in terms of its desired properties, without constructing and explicit model.

Methods

Model oriented methods
- Z
- VDM (Vienna Development Method)

Algebaric methods
- Clear
- OBJ
- ACT ONE

- This distinction between model-oriented and property oriented methods is not as clear-cut as it might at first appear; in practice, Z specifications often describe certain aspects of systems by giving axioms which must be satisfied by the system, and this amounts to a property-oriented specification.
- Algebaric specifications often describe a collection of a basic data-types in a property oriented way, then use these to build a model of the system being specified.

Model-oriented methods

- The closest method to Z is the "Vienna Development Method (VDM)", which originated at the IBM Vienna Laboratory, and has been developed in the work of Dines Bjorner and Cliff Jones.

• There are two elementary kinds of declaration:

1- One of these introduces a single variable, the type being given by a set-valued term.

2- The other introduces all the variables of a schema by means of a schema designator (SDES).

Operations in VDM-SL

In VDM-SL an operation consists of four sections:

- the operation header
- the external clause
- the precondition
- the postcondition

Components of VDM-SL

As we studied, the components of a complete VDM-SL specification are:

- Declaration of constants
- State definition
- Functions
- Operations

Set Operations

- ***Three operations that take two sets as input and return a new set as output***
  - ***Set union***
  - ***Set intersection***
  - ***Set difference***

- A specification consists of a sequence of definitions, which are of **three** kinds:

-> Definitions of global-set names

-> Definition of global variables

-> Definition of schemas

Model oriented methods

- Z
- VDM (Vienna Development Method)

Algebaric methods

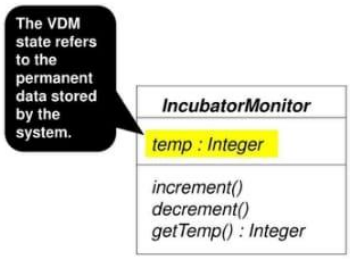
- Clear
- OBJ
- ACT ONE

6:40 69

instrinic

Specifying the State

- The first thing we will consider for our formal specification is what is known as the state of the system.
- In VDM-SL the state refers to the permanent data that must be stored by the system, and which can be accessed by means of operations.



13

Intrinsic types available in VDM-SL

N: natural numbers (positive whole numbers)

N_1: natural numbers excluding zero

Z: integers (positive and negative whole numbers)

R: real numbers (positive and negative numbers that can include a fractional part)

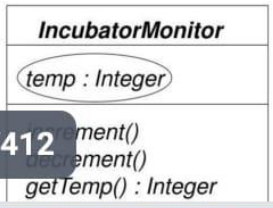
B: boolean values (TRUE or FALSE)

Char: the set of alphanumeric characters

14

Specifying the state of Incubator Monitor System

UML **VDM-SL**




```
state IncubatorMonitor of
    temp : Z
end
```

41/412

Chamet